

PROGRAMACIÓN LINEAL

Análisis y Diseño de Datos y Algoritmos
Apuntes creados por David Romero
30/07/2019 – 1.0



Atribución-NoComercial-CompartirIgual
4.0 Internacional (CC BY-NC-SA 4.0)

PROGRAMACIÓN LINEAL

1. Optimizar un problema
2. Algoritmo simplex
3. Problemas de programación lineal
4. Implementación en Java
5. Errores típicos
6. Ejercicios resueltos



PROGRAMACIÓN LINEAL

1. Optimizar un problema
2. Algoritmo simplex
3. Problemas de programación lineal
4. Implementación en Java
5. Errores típicos
6. Ejercicios resueltos

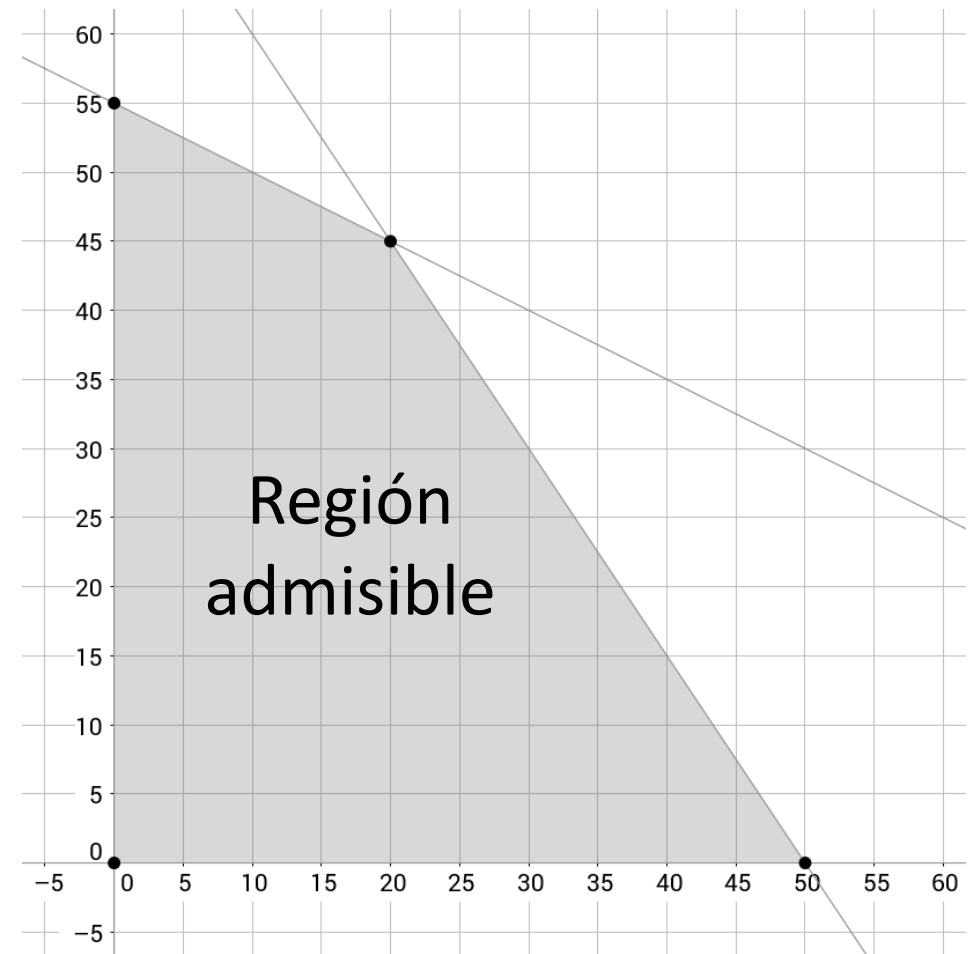


1.1. ¿Qué es optimizar un problema?

Definimos la **optimización de un problema** como la solución del mismo teniendo en cuenta un **conjunto de restricciones** y una **función objetivo**.

Se busca que la solución **maximice** (o minimice) el valor de la función objetivo.

En ocasiones, la complejidad del algoritmo que optimiza el problema es demasiado alta, siendo necesario el uso de algoritmos de aproximación (*Algoritmos Genéticos*).



1.2. Función objetivo

Se define la función objetivo de un problema de optimización como

$$\min_{x \in \Omega} f(x)$$

Donde x es un vector de n variables que deben tomar valores en un dominio Ω en concreto.

1.3. Transformación minimización - maximización

Un problema de maximización puede transformarse a un problema de minimización haciendo

$$\max_{x \in \Omega} f(x) = \min_{x \in \Omega} -f(x)$$

Donde x es un vector de n variables que deben tomar valores en un dominio Ω en concreto.

1.4. Problemas multiobjetivos

A veces, es necesario optimizar varios objetivos, quedando un problema multiobjetivos

$$\min_{x \in \Omega} (f_1(x), \dots, f_m(x))$$

Esto puede ser convertido a un problema monoobjetivo

$$\min_{x \in \Omega} \sum_{i=1}^m \omega_i f_i \quad \sum_{i=1}^m \omega_i = 1$$

1.5. Problema de Programación Lineal

Transformar un problema en un conjunto de restricciones lineales se dice que es un **Problema de Programación Lineal (PL)**

Cuando las variables que entran en juego solo pueden tomar valores enteros en un rango, decimos que es un **Problema de Programación Lineal Entera (PLE)**

PROGRAMACIÓN LINEAL

1. Optimizar un problema
- 2. Algoritmo simplex**
3. Problemas de programación lineal
4. Implementación en Java
5. Errores típicos
6. Ejercicios resueltos



2.1. Algoritmo simplex

Un algoritmo Simplex no es más que un conjunto de métodos matemáticos que permiten resolver problemas de programación lineal.

Generalmente, en estos problemas se busca **satisfacer** la **función objetivo** sobre un **conjunto de inecuaciones lineales**.

Este algoritmo fue desarrollado por el matemático norteamericano George Dantzig (1914-2005). Hay multitud de problemas que se pueden resolver mediante este algoritmo.

Más información:

https://es.wikipedia.org/wiki/Algoritmo_simplex

PROGRAMACIÓN LINEAL

1. Optimizar un problema
2. Algoritmo simplex
- 3. Problemas de programación lineal**
4. Implementación en Java
5. Errores típicos
6. Ejercicios resueltos

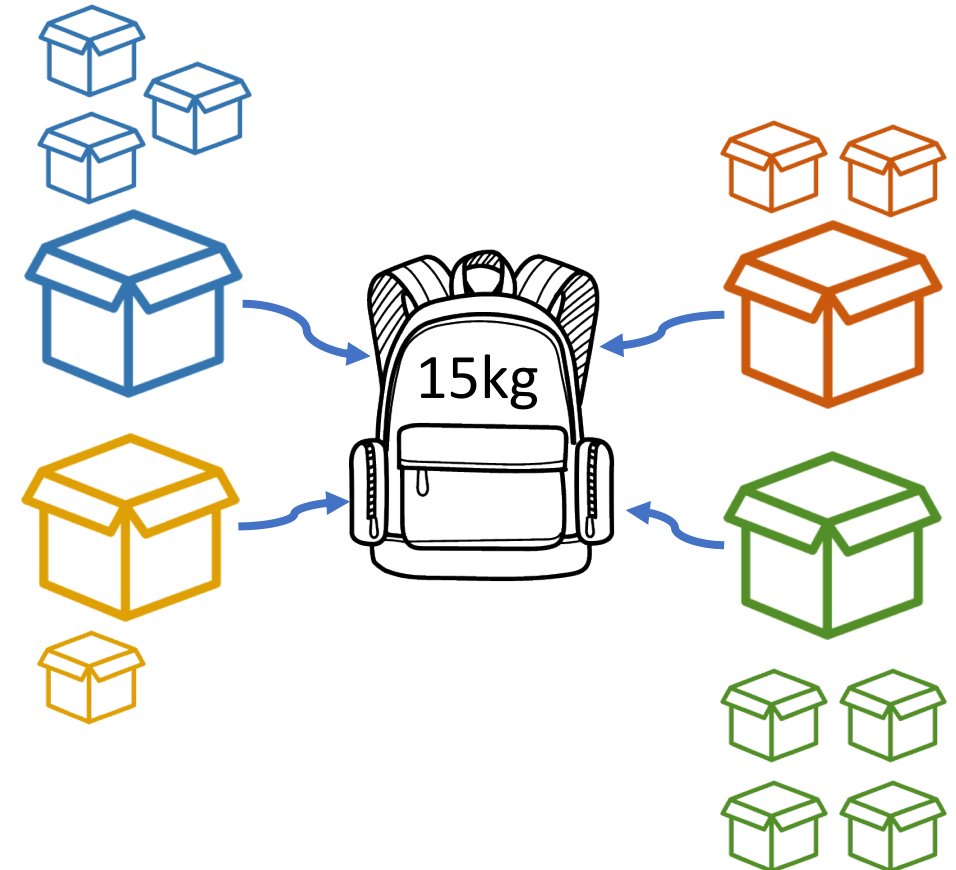


3.1. El problema de la mochila

Imagina que eres un vendedor de joyas preciosas y extraordinarias, y tienes que avanzar a través de un largo camino por el bosque para llegar al pueblo y venderlas.

Pero tienes un problema: debido a tu situación económica y a problemas de salud y de espalda derivados de tu trabajo, posees una mochila con la que no puedes llevar más de 15 kg de peso. A ese problema le añadimos otro: las joyas van en unas pequeñas cajas, que tienen su peso y su valor, como es lógico. **Deseo maximizar el beneficio.**

¿Qué cajas elijo?



3.1. El problema de la mochila

Variables

- Sea C la capacidad de la mochila
- Sea L una lista de cajas de tamaño n
- Sea m una lista de enteros, donde cada entero m_i es el número de veces que se repite cada caja de la lista anterior (o, también, el número de tipos que existen de esa caja)
- Como cada caja posee un peso y un valor, definimos **obi** como un objeto (caja) que contiene un peso w_i y un valor v_i , siendo i el tipo de la caja (no confundir con el número)
- El número de cajas de tipo i será x_i

3.1. El problema de la mochila

Función objetivo

Si quiero maximizar el beneficio, está claro que entra el juego el valor de las cajas, luego:

$$\max \sum_{i=0}^{n-1} x_i v_i$$

3.1. El problema de la mochila

Restricciones

La función objetivo tiene una pega; la suma del peso de todas las cajas no puede exceder la capacidad de la mochila, luego:

$$\sum_{i=0}^{n-1} x_i w_i \leq C$$

¡No es la única restricción! Si tengo 4 cajas de tipo A, no puede ser que una solución sea que la mochila tenga 6 cajas de ese tipo, es imposible, luego:

$$x_i \leq m_i, \quad i \in [0, n - 1]$$

3.1. El problema de la mochila

Restricciones

No puede haber un número no entero de cajas.

*Si es un problema de programación lineal **entera**, el número de cajas de la solución debe ser eso, un número entero, luego:*

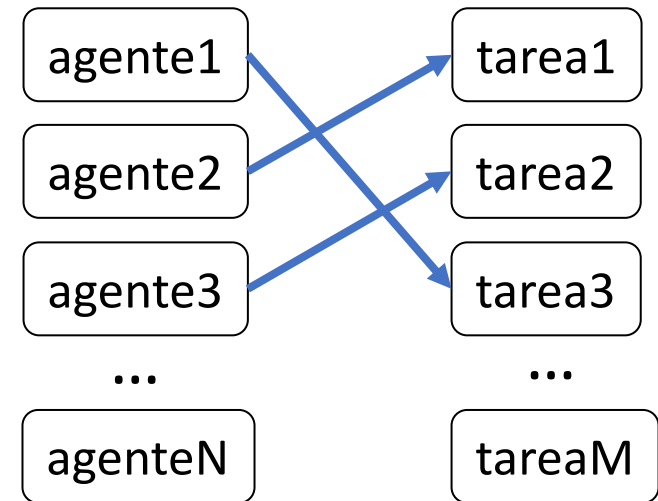
$$\text{int } x_i, \quad i \in [0, n - 1]$$

3.2. El problema de la asignación de tareas

Hay un número de agentes y un número de tareas. Cualquier agente puede ser asignado para desarrollar cualquier tarea, contrayendo algún coste que puede variar dependiendo del agente y la tarea asignados.

Es necesario, para desarrollar todas las tareas, asignar un solo agente a cada tarea de modo que el coste total de la asignación sea mínimo.

¿Cuál es el emparejamiento óptimo?

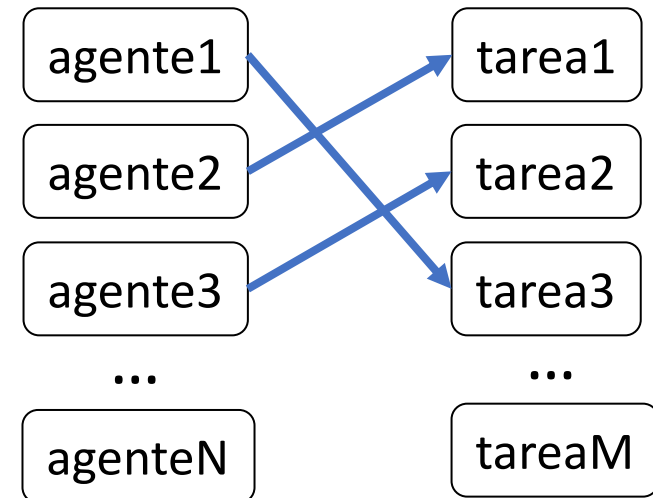


3.2. El problema de la asignación de tareas

Variables

- Sea ***La*** la lista de agentes
- Sea ***Lt*** la lista de tareas
- Sea ***n*** el tamaño de las listas (tienen el mismo tamaño)
- Sea ***c_{ij}*** el coste de asignar al agente *i* la tarea *j*
- Sea ***x_{ij}*** una variable binaria, tal que

$$x_{ij} = \begin{cases} 1 & \text{si el agente } i \text{ ejecuta la tarea } j \\ 0 & \text{en cualquier otro caso} \end{cases}$$

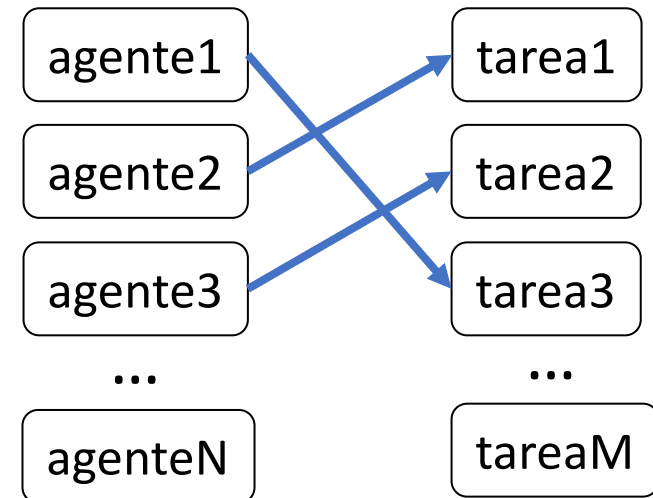


3.2. El problema de la asignación de tareas

Función objetivo

¿Qué asignación hago para que el coste total sea mínimo?

$$\min \sum_{i=0, j=0}^{n-1, n-1} x_{ij} c_{ij}$$



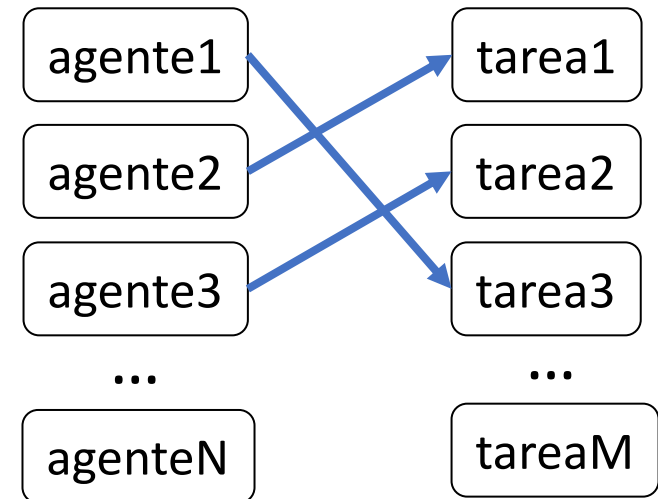
3.2. El problema de la asignación de tareas

Restricciones

La regla es 1 agente = 1 tarea. Si todas las asignaciones (binarias) se realizan, la "suma" debe ser 1 (todas están a true). No puede haber tareas sin asignar, es una función biyectiva.

$$\sum_{j=0}^{n-1} x_{ij} = 1, \quad i \in [0, n-1]$$

$$\sum_{i=0}^{n-1} x_{ij} = 1, \quad j \in [0, n-1]$$

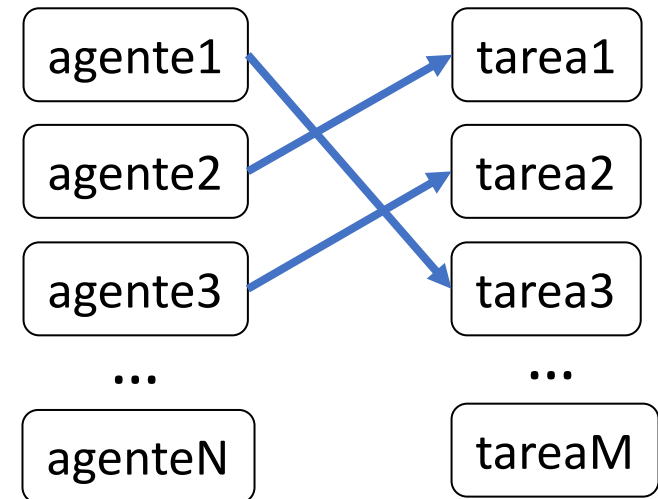


3.2. El problema de la asignación de tareas

Restricciones

x_{ij} es una variable binaria. Además, el agente i -ésimo y la tarea j -ésima deben estar dentro del rango.

$$0 \leq x_{ij} \leq 1, \quad i \in [0, n - 1], \quad j \in [0, n - 1]$$



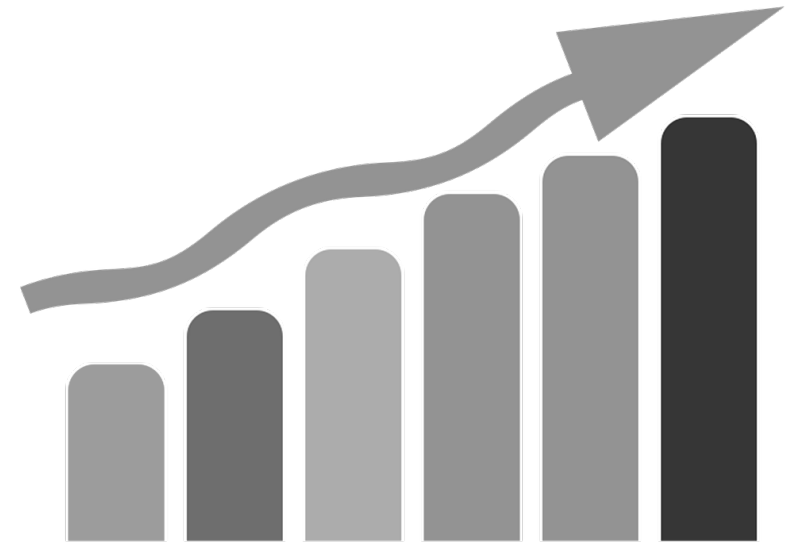
3.3. El problema de la inversión de capital

Dispongo de una cierta cantidad para invertir. Mi asesor fiscal me comenta que puedo invertir en varias oportunidades, no cuando yo quiera, y que cada oportunidad me pedirá una cantidad y un posible resultado, que espero que sea favorable.

El asesor me comenta que solo puedo hacer 2 inversiones como máximo y siguiendo estas reglas:

- Si realizo la inversión r , me obliga a hacer también la s
- Si realizo la inversión u , no me deja hacer la inversión v y viceversa

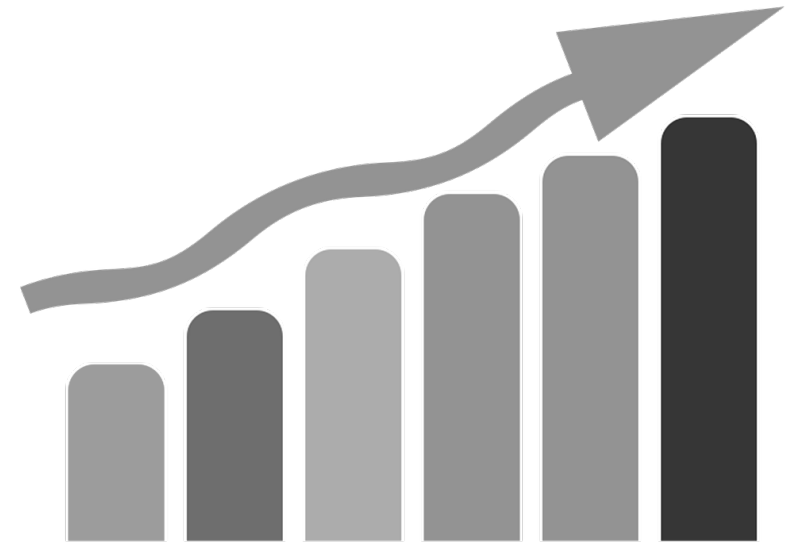
¿Qué 2 inversiones hago?



3.3. El problema de la inversión de capital

Variables

- Sea T la cantidad de dinero a invertir
- Sea n el número de oportunidades que tengo para invertir
- Sea i el número de una inversión, se define c_i como la cantidad que voy a invertir, v_i como el beneficio que espero obtener, y x_i una inversión en concreto

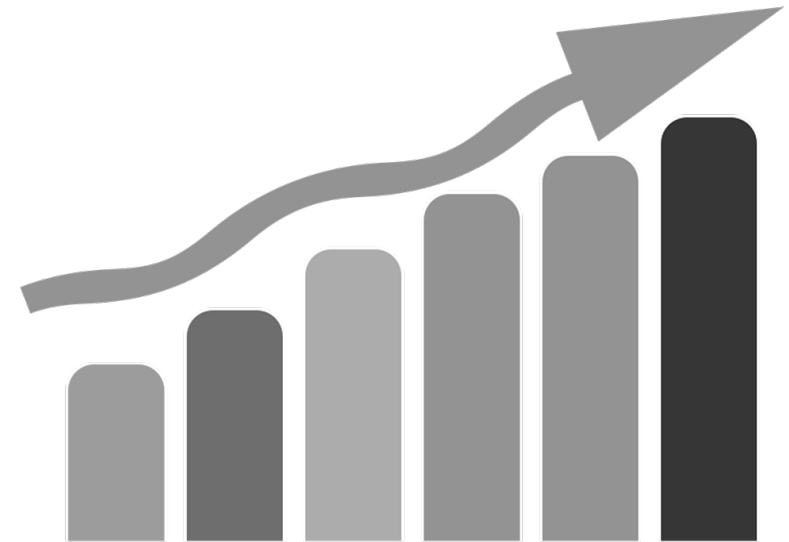


3.3. El problema de la inversión de capital

Función objetivo

¿Qué inversiones realizo para que el beneficio sea máximo?

$$\max \sum_{i=0}^{n-1} x_i v_i$$



3.3. El problema de la inversión de capital

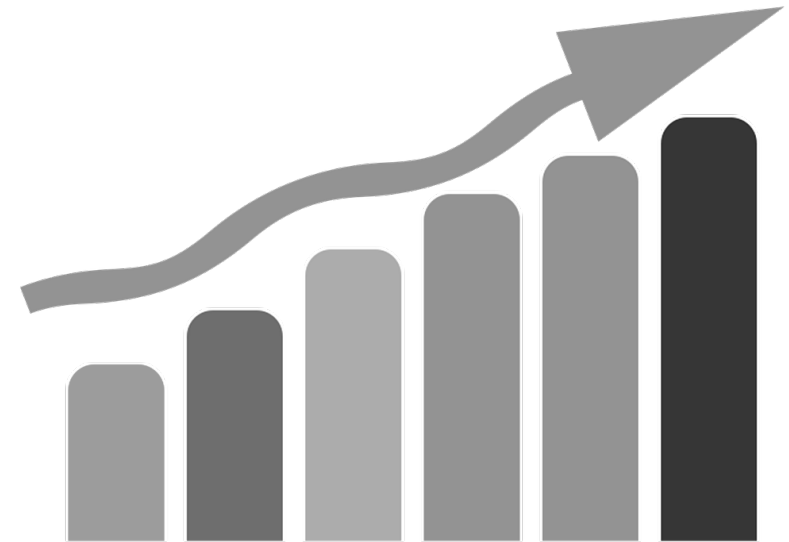
Restricciones

La cantidad invertida no puede ser superior a la total, es imposible

$$\sum_{i=0}^{n-1} x_i c_i \leq T$$

Solo puedo realizar hasta un máximo de 2 inversiones

$$\sum_{i=0}^{n-1} x_i \leq 2$$



3.3. El problema de la inversión de capital

Restricciones

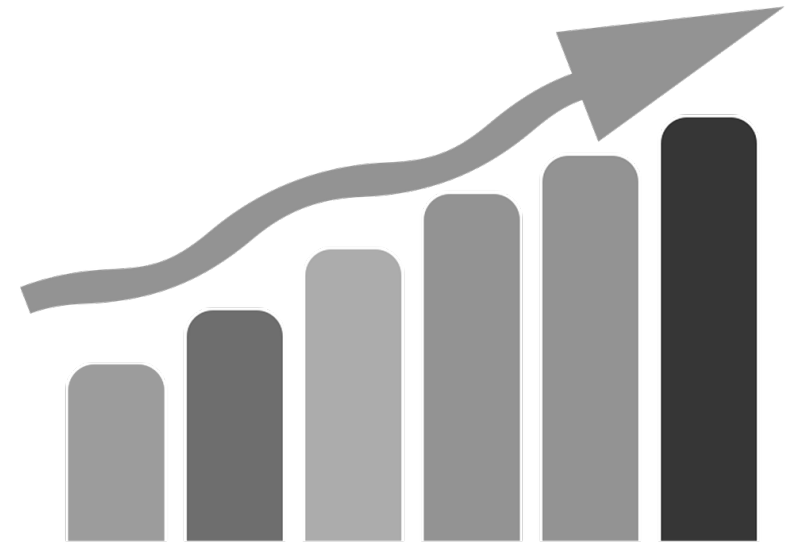
Si realizo la inversión r , me obliga a hacer también la s

$$x_r - x_s \leq 0$$

(También es válido $x_r + x_s > 1$)

Si realizo la inversión u , no me deja hacer la inversión v y viceversa

$$x_u + x_v \leq 1$$



3.4. El problema de recubrimiento de conjuntos

También llamado *problema del conjunto de cobertura* (*Set Covering*), consiste en lo siguiente:

Dado un conjunto de elementos $\{1, 2, \dots, m\}$ (el *universo*) y n conjuntos cuya unión comprende el universo, se trata de encontrar el menor número de conjuntos cuya unión contiene todos los elementos del universo.

$$U = \{1, 2, 3, 4, 5\}$$

$$S = \{\{1, 2, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\}\}.$$

$$\{\{1, 2, 3\}, \{4, 5\}\}$$

cubren todos los elementos de U

3.4. El problema de recubrimiento de conjuntos

Variables

- Sea U un universo
- Sea S un conjunto de m conjuntos
- Sea j cada conjunto de S
- Sea w_j el peso de un conjunto j
- Sea i el índice de cada subconjunto del conjunto S .
- Sea e_i un elemento del conjunto i
- Sea $\varphi(i) = \{j: e_i \in s_j\}$, es decir, el conjunto de índices de conjuntos que incluyen al elemento e_i
- Sea x_j una variable binaria, tal que

$$x_j = \begin{cases} 1 & \text{si el conjunto } j \text{ es escogido} \\ 0 & \text{en cualquier otro caso} \end{cases}$$

$$U = \{1,2,3,4,5\}$$

$$S = \{\{1,2,3\}, \{2,4\}, \{3,4\}, \{4,5\}\}.$$

$$\{\{1,2,3\}, \{4,5\}\}$$

cubren todos los elementos de U

3.4. El problema de recubrimiento de conjuntos

Función objetivo

¿Qué número mínimo de conjuntos elijo tal que la unión de todos sea U ?

$$\min \sum_{j=0}^{m-1} w_j x_j$$

$$U = \{1,2,3,4,5\}$$

$$S = \{\{1,2,3\}, \{2,4\}, \{3,4\}, \{4,5\}\}.$$

$$\{\{1,2,3\}, \{4,5\}\}$$

cubren todos los elementos de U

3.4. El problema de recubrimiento de conjuntos

Restricciones

La unión de los conjuntos elegidos debe ser el universo

$$\sum_{j:\varphi(i)} x_j \geq 1, \quad i \in [0, m)$$

$$U = \{1,2,3,4,5\}$$

$$S = \{\{1,2,3\}, \{2,4\}, \{3,4\}, \{4,5\}\}.$$

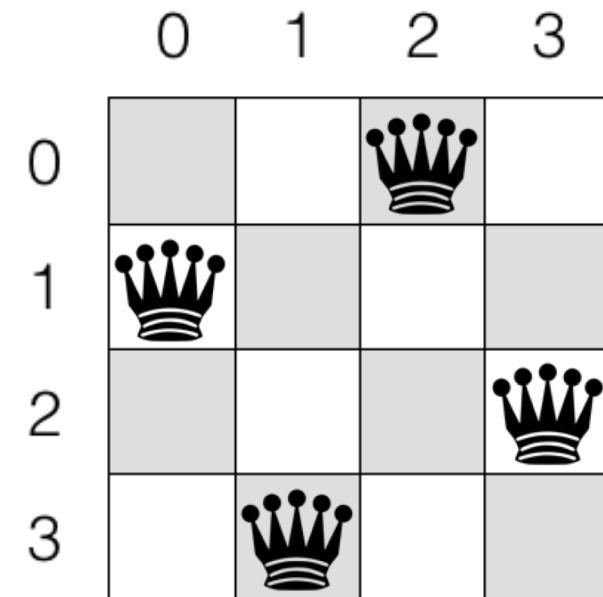
$$\{\{1,2,3\}, \{4,5\}\}$$

cubren todos los elementos de U

3.5. El problema de las N-Reinas

¿Es posible colocar n reinas en un tablero de ajedrez de $n \times n$ de tal forma que ninguna de ellas amenace al resto?

Recuerda que la reina del ajedrez puede moverse en horizontal, en vertical y en diagonal.

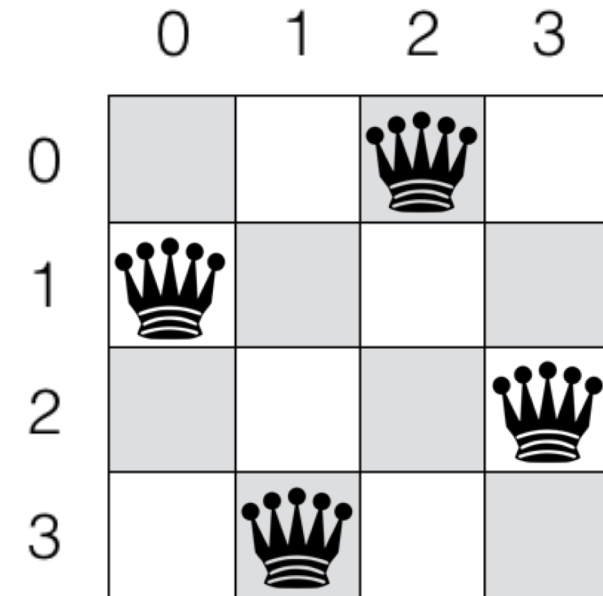


3.5. El problema de las N-Reinas

Variables

- Sea i una fila
- Sea j una columna
- Sea $d1$ la diagonal principal
- Sea $d2$ la diagonal secundaria
- Sea $\varphi(d1)$ el conjunto de casillas de la diagonal principal
- Sea $\varphi(d2)$ el conjunto de casillas de la diagonal secundaria
- Sea x_{ij} una variable binaria, tal que

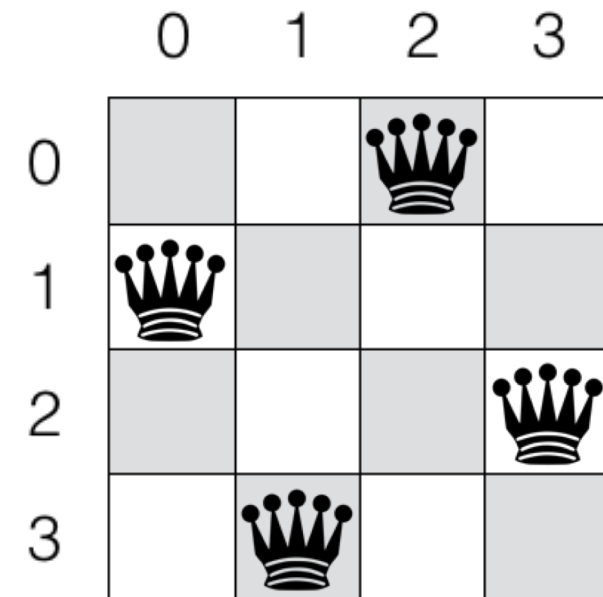
$$x_{ij} = \begin{cases} 1 & \text{si se coloca una reina en la casilla } ij \\ 0 & \text{en cualquier otro caso} \end{cases}$$



3.5. El problema de las N-Reinas

Función objetivo

No hay, realmente con que cumpla las restricciones lineales es YA UNA SOLUCIÓN del problema.



3.5. El problema de las N-Reinas

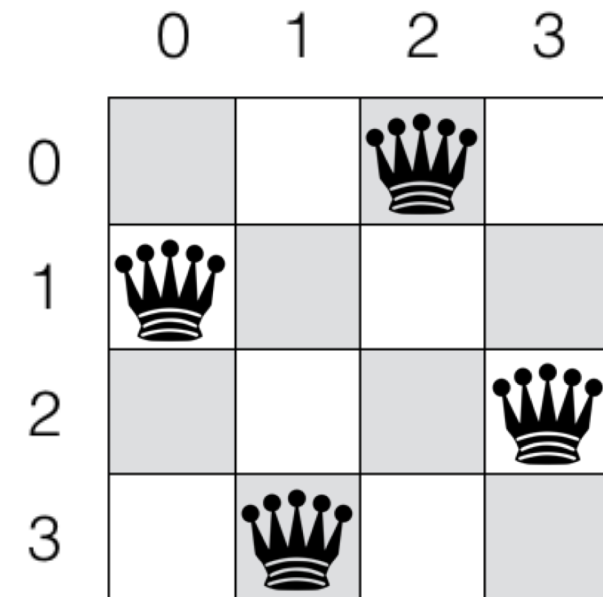
Restricciones

Tiene que haber una reina (y solo una) en cada fila

$$\sum_{i=0}^{n-1} x_{ij} = 1, \quad j = 0, \dots, n-1$$

Tiene que haber una reina (y solo una) en cada columna

$$\sum_{j=0}^{n-1} x_{ij} = 1, \quad i = 0, \dots, n-1$$



3.5. El problema de las N-Reinas

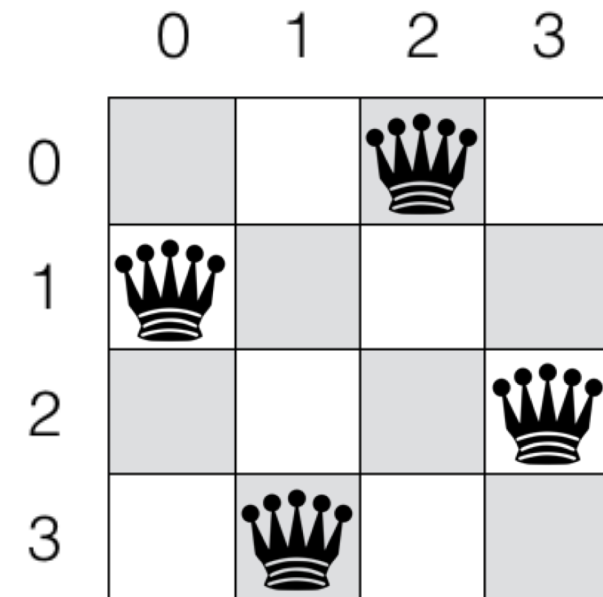
Restricciones

Puede no haber ninguna reina en la diagonal principal, pero de existir, solo puede haber una

$$\sum_{(i,j): \varphi(d1)} x_{ij} \leq 1, \quad d1 = -(n-1), \dots, n-1$$

Puede no haber ninguna reina en la diagonal secundaria, pero de existir, solo puede haber una

$$\sum_{(i,j): \varphi(d2)} x_{ij} \leq 1, \quad d2 = 0, \dots, 2n-2$$



3.6. El problema del Sudoku

El problema consiste en rellenar con los enteros $[1 \dots n]$ las casillas de un tablero $n \times n$ de tal manera que cada fila, cada columna y cada subtabla incluyan una vez todos los enteros del 1 al n .

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

3.6. El problema del Sudoku

Variables

- Sea f el número de filas
- Sea c el número de columnas
- Sea $n = f * c$ el número de elementos
- Sea v un valor comprendido entre 1 y n
- Sea i una fila
- Sea j una columna
- Sea t el número de cada subtabla, de forma que

$$t = \frac{j}{c}c + \frac{i}{f}$$

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

3.6. El problema del Sudoku

Variables

- Sea $\varphi(t)$ el conjunto de casillas de la subtabla t
- Sea x_{ijv} una variable binaria, tal que

$$x_{ijv} = \begin{cases} 1 & \text{si la casilla } ij \text{ toma el valor } v \\ 0 & \text{en cualquier otro caso} \end{cases}$$

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

3.6. El problema del Sudoku

Función objetivo

No hay, realmente con que cumpla las restricciones lineales es
YA UNA SOLUCIÓN del problema.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

3.6. El problema del Sudoku

Restricciones

Todas las filas deben contener los valores de 1 a n

$$\sum_{i=0}^{n-1} x_{ijv} = 1, \quad j, v = 0, \dots, n-1$$

Todas las columnas deben contener los valores de 1 a n

$$\sum_{j=0}^{n-1} x_{ijv} = 1, \quad i, v = 0, \dots, n-1$$

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

3.6. El problema del Sudoku

Restricciones

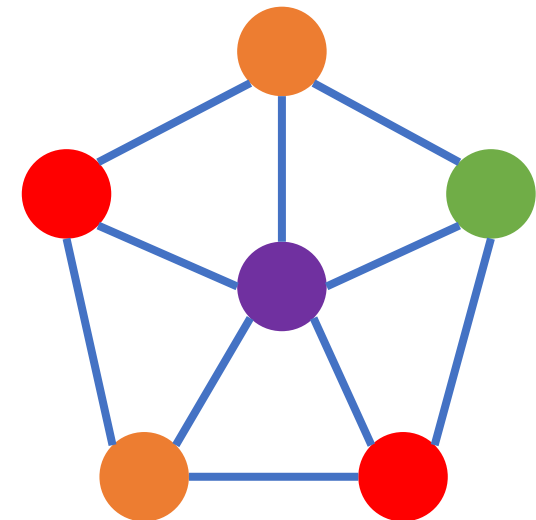
Todas las subtablas deben contener los valores de 1 a n

$$\sum_{(i,j):\varphi(t)} x_{ijv} = 1, \quad t, v = 0, \dots, n-1$$

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

3.7. El problema del coloreado de un grafo

Dado un grafo, queremos buscar el mínimo número de colores tal que, dando un color a cada vértice, todos sus vértices adyacentes tengan distinto color.



3.7. El problema del coloreado de un grafo

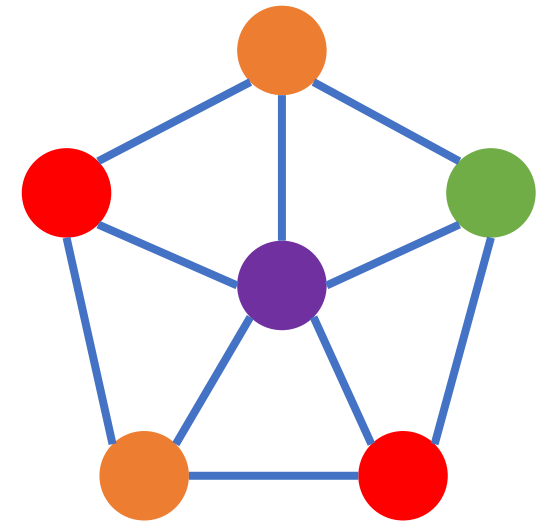
Variables

- Sea n el número de vértices
- Sea k un color
- Sea i un vértice
- Sea j un vértice adyacente a i
- Sea x_{ik} una variable binaria, tal que

$$x_{ik} = \begin{cases} 1 & \text{si al vértice } i \text{ se le asigna el color } k \\ 0 & \text{en cualquier otro caso} \end{cases}$$

- Sea y_k una variable binaria, tal que

$$y_k = \begin{cases} 1 & \text{si el color } k \text{ es usado} \\ 0 & \text{en cualquier otro caso} \end{cases}$$

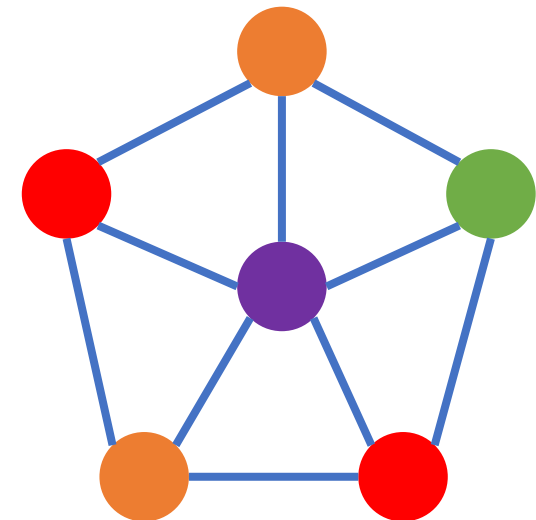


3.7. El problema del coloreado de un grafo

Función objetivo

¿Cuál es el menor número de colores que tengo que usar?

$$\min \sum_{k=0}^{n-1} y_k$$



3.7. El problema del coloreado de un grafo

Restricciones

Todos los vértices tienen que tener un color

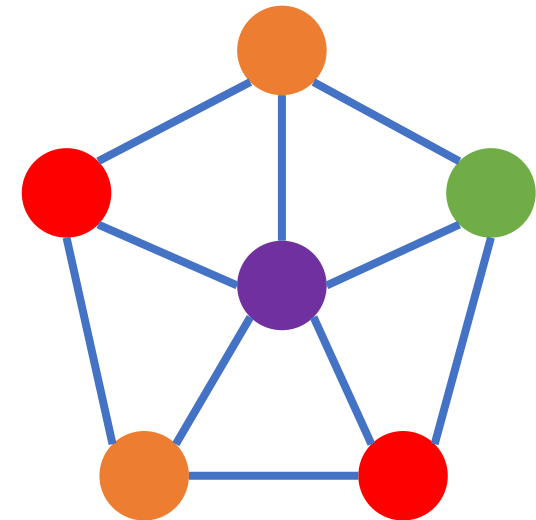
$$\sum_{k=0}^{n-1} x_{ik} = 1, \quad i = 0, \dots, n-1$$

Si un vértice i recibe el color k , éste se utiliza

$$x_{ik} - y_k \leq 0, \quad i, k = 0, \dots, n-1$$

Si los vértices i, j son adyacentes, no pueden tener el mismo color

$$x_{ik} + x_{jk} \leq 1, \quad i, k = 0, \dots, n-1$$



PROGRAMACIÓN LINEAL

1. Optimizar un problema
2. Algoritmo simplex
3. Problemas de programación lineal
- 4. Implementación en Java**
5. Errores típicos
6. Ejercicios resueltos



4.1. Implementación PL

En la asignatura se proporcionan dos algoritmos para trabajar:

- **AlgoritmoPL:** Implementa método Simplex (*librerías de Apache*)
- **AlgoritmoPLI:** Implementa método de Programación Lineal Entera (*librerías de LPSolve*).

En realidad, nosotros usaremos siempre AlgoritmoPLI ya que éste incluye al anterior.

4.2. Ejemplo del problema de inversión de capital (I)

Deseamos invertir una cantidad total de $T = 15$ en $n = 5$ oportunidades de inversión.

Oportunidad de inversión	Capital requerido	Beneficio esperado
0	10	16
1	4	6
2	5	7
3	7	0
4	2	3

¿En qué oportunidades invierto?

4.2. Ejemplo del problema de inversión de capital (I)

Deseamos invertir una cantidad total de $T = 15$ en $n = 5$ oportunidades de inversión.

Fichero en formato LpSolve

```
/* función objetivo */
max: 16.0x0 +6.0x1 +7.0x2 +10.0x3 +3.0x4;

/* restricciones lineales */
10x0 + 4x1 + 5x2 + 7x3 + 2x4 <= 15;
x0 <= 1;
x1 <= 1;
x2 <= 1;
x3 <= 1;
x4 <= 1;

/* tipos de las variables */
int x0,x1,x2,x3,x4;
```

Oportunidad de inversión	Capital requerido	Beneficio esperado
0	10	16
1	4	6
2	5	7
3	7	0
4	2	3

4.2. Ejemplo del problema de inversión de capital (I)

El esquema es siempre el mismo:

- **Función objetivo de tipo *max* o *min***
- **Restricciones lineales**
- **Tipos de las variables usadas**

Varios detalles respecto al problema anterior:

- *No me han dicho que el capital requerido tenga que ser exactamente el que me piden, puedo invertir menos*
($x_0 \leq 1.0$).
- El total del capital de inversión no puede ser superior a 15
($x_0 + x_1 + x_2 + x_3 + x_4 \leq 15.0$)

4.2. Ejemplo del problema de inversión de capital (II)

¿Y si complicamos el problema?

- Sólo se pueden hacer 2 inversiones como máximo.
- Si se hace la inversión 2, la 3 también se debe hacer.
- Si se hace la inversión 0, la 1 no se puede hacer, y viceversa.

Oportunidad de inversión	Capital requerido	Beneficio esperado
0	10	16
1	4	6
2	5	7
3	7	0
4	2	3

4.2. Ejemplo del problema de inversión de capital (II)

Deseamos invertir una cantidad total de $T = 15$ en $n = 5$ oportunidades de inversión.

Fichero en formato LpSolve

```
/* función objetivo */
max: 16.0x0+6.0x1+7.0x2+10.0x3+3.0x4;

/* restricciones lineales */
10.0x0+4.0x1+5.0x2+7.0x3+2.0x4<=15.0;
x0+x1+x2+x3+x4<=2;
x2-x3<=0;
x0+x1<=1;

/* tipos de las variables */
bin x0,x1,x2,x3,x4;
```

Oportunidad de inversión	Capital requerido	Beneficio esperado
0	10	16
1	4	6
2	5	7
3	7	0
4	2	3

4.3. Implementación Algoritmo PLI

Si el fichero LpSolve es viable crearlo a mano...

Si es posible modelar el problema a mano porque el conjunto de variables es pequeño, creamos un fichero con el formato *LpSolve* y, posteriormente, lo usaremos para crear un algoritmo de tipo *PLI*.

```
AlgoritmoPLI a =  
Algoritmos.createPLI("nombredelfichero");  
a.ejecuta();
```

4.3. Implementación Algoritmo PLI

Si el fichero LpSolve es inviable crearlo a mano...

Si no es posible crear el fichero a mano, tiraremos del método *getConstraints()*, que generará una cadena con el mismo formato que un fichero *LpSolve*

```
AlgoritmoPLI a = AlgoritmoPLI.create();  
a.setConstraints(r);  
a.ejecuta();
```

Sea *r* una cadena de tipo *LpSolve*

PROGRAMACIÓN LINEAL

1. Optimizar un problema
2. Algoritmo simplex
3. Problemas de programación lineal
4. Implementación en Java
- 5. Errores típicos**
6. Ejercicios resueltos



5.1. Saltos de línea

No poner el (;) al final de cada línea
¡Por favor, separa las líneas!

5.2. Restricciones que no existen

Es muy típico pensar en restricciones del estilo “*la suma de los elementos 1 y 2 no puede ser 5*” y escribirlo así:

$$x_1 + x_2 \neq 5;$$

Bien, pues esto es imposible hacerlo así, no está permitido. Mejor usa alternativas como ($<$), ($>$), etc.

5.3. Combinaciones no lineales

Si usas líneas como

$c_0 x_0$;

Asegúrate que una de las dos variables sea **un número constante**. Precisamente un problema de programación **lineal** se llama así **porque sus restricciones son lineales**.

El producto de dos variables con valores no constantes lo convierte en un problema de programación **NO** lineal.

5.4. Usar el operador (*) para expresar el producto, o dejar un espacio en blanco

Si quiero expresar el producto entre dos variables, deja un espacio en blanco, tal que así:

`c0 x0;`

En caso de que sepas el valor de la constante, no es necesario dejar el espacio en blanco:

`10x1;`

5.5. Usar más de una función objetivo

Las siguientes líneas:

max: $x_1 + x_2 + x_3 \leq 10$;

max: $x_4 + x_5 \leq 5$;

Son erróneas, solo puede haber una función objetivo.

Mejor usa:

max: $x_1 + x_2 + x_3 + x_4 + x_5 \leq 15$;

5.6. No seguir el orden estricto

El orden en el fichero LpSolve tiene que ser **SIEMPRE** el mismo:

- 1) Función objetivo de tipo max o min
- 2) Restricciones lineales
- 3) Tipos de las variables usadas

PROGRAMACIÓN LINEAL

1. Optimizar un problema
2. Algoritmo simplex
3. Problemas de programación lineal
4. Implementación en Java
5. Errores típicos
6. Ejercicios resueltos



6.1. La panadería

Una panadería dispone de una cantidad de kilos de harina y litros de agua para elaborar sus distintos tipos de productos. Teniendo en cuenta que cada tipo de producto reporta un beneficio distinto, y que cada uno requiere una cantidad distinta de harina y agua para fabricarse, es necesario determinar **cuántas unidades** elaborar de cada uno de los productos de forma que el **beneficio sea el máximo**.

Por ejemplo, para 50 kilos de harina y 40 litros de agua, la mejor opción sería

- 35 unidades de P1
- 54 unidades de P2
- 0 unidades de P3
- 4 unidades de P4

Tipo de producto	Kg de harina por unidad	Litros de agua por unidad	Beneficio por unidad
P1	0,4	0,6	2
P2	0,6	0,3	1,5
P3	0,5	0,4	1
P4	0,7	0,7	2,5

6.1. La panadería (implementando archivo LpSolve)

```
max: B;  
B = 2p1 + 1.5p2 + 1p3 + 2.5p4;  
  
H >= 0.4p1 + 0.6p2 + 0.5p3 + 0.7p4;  
A >= 0.6p1 + 0.3p2 + 0.4p3 + 0.7p4;  
  
p1 >= 0;  
p2 >= 0;  
p3 >= 0;  
p4 >= 0;  
  
H = 50;  
A = 40;  
int p1, p2, p3, p4;
```

Producto

```
getKgHarinaPorUnidad(): Double  
getLitrosAguaPorUnidad(): Double  
getBeneficioPorUnidad(): Double
```

ProblemaPanaderia

```
numProductos: Integer  
productos: List<Producto>  
getKgHarinaPorProducto(): Double  
getLitrosAguaPorProducto(): Double  
getBeneficioPorUnidad(): Double
```

Producto	Harina	Agua	Beneficio
P1	0,4	0,6	2
P2	0,6	0,3	1,5
P3	0,5	0,4	1
P4	0,7	0,7	2,5

6.1. La panadería (generando archivo LpSolve)

```
private static String getConstraints() {
    String res = "max: B;\n";
    res += IntStream.range(0,
        ProblemaPanaderia.numProductos)
        .boxed()
        .map(i -> String.format("%sp%d",
            ProblemaPanaderia.getBeneficioPorUnidad(i)
            .toString().replace(",", "."), i))
        .collect(Collectors.joining("+", "B = ",
            ";\n"));
    res += "\n";
    res += IntStream.range(0,
        ProblemaPanaderia.numProductos)
        .boxed()
        .map(i -> String.format("%sp%d",
            ProblemaPanaderia.getKgHarinaPorProducto(i)
            .toString().replace(",", "."), i))
        .collect(Collectors.joining("+", "H >= ",
            ";\n"));
}
```

Producto

```
getKgHarinaPorUnidad(): Double
getLitrosAguaPorUnidad(): Double
getBeneficioPorUnidad(): Double
```

ProblemaPanaderia

```
numProductos: Integer
productos: List<Producto>
getKgHarinaPorProducto(): Double
getLitrosAguaPorProducto(): Double
getBeneficioPorUnidad(): Double
```

Producto	Harina	Agua	Beneficio
P1	0,4	0,6	2
P2	0,6	0,3	1,5
P3	0,5	0,4	1
P4	0,7	0,7	2,5

6.1. La panadería (generando archivo LpSolve)

```

    . . .
    res += IntStream.range(0,
        ProblemaPanaderia.numProductos)
        .boxed()
        .map(i -> String.format("%sp%d",
            ProblemaPanaderia.getLitrosAguaPorProducto(i)
                .toString().replace(",", "."), i))
                .collect(Collectors.joining("+", "A >= ",
                    ";\\n")));

    res += "H = " + ProblemaPanaderia.kgHarina + ";\\n";
    res += "A = " + ProblemaPanaderia.litrosAgua + ";\\n";

    res += IntStream.range(0,
        ProblemaPanaderia.numProductos)
        .boxed()
        .map(i -> String.format("p%d", i))
        .collect(Collectors.joining(",", "int ", ";\\n"));

    return res;

}

```

Producto

```

getKgHarinaPorUnidad(): Double
getLitrosAguaPorUnidad(): Double
getBeneficioPorUnidad(): Double

```

ProblemaPanaderia

```

numProductos: Integer
productos: List<Producto>
getKgHarinaPorProducto(): Double
getLitrosAguaPorProducto(): Double
getBeneficioPorUnidad(): Double

```

Producto	Harina	Agua	Beneficio
P1	0,4	0,6	2
P2	0,6	0,3	1,5
P3	0,5	0,4	1
P4	0,7	0,7	2,5

Miguel Toro

“Tema 16. Algoritmos de aproximación”, del libro “Introducción a la programación”

Departamento de Lenguajes y Sistemas Informáticos

Universidad de Sevilla

“Tema 5. Algoritmos de aproximación”, de la asignatura “Análisis y diseño de datos y algoritmos”

Departamento de Lenguajes y Sistemas Informáticos

Escuela Técnica Superior de Ingeniería Informática

Universidad de Sevilla, 2017-2018

“Práctica 11. Programación lineal y programación lineal entera”, de la asignatura “Análisis y diseño de datos y algoritmos”

Departamento de Lenguajes y Sistemas Informáticos

Escuela Técnica Superior de Ingeniería Informática

Universidad de Sevilla, 2017-2018